

*Same as well
Just better*

SAME YOU SINCE 2009
JUST BETTER

Before we start

Let's talk about your HW

- How was the Merge Conflict simulation?
- Did anyone struggle with the Pull Request flow?
- The Golden Rule: Why don't we merge directly to main?

Lesson 5

Python for DevOps Part 1 Foundations, Syntax, and Scripting Logic

The Man, The Myth The **LEGEND** HIMSELF

Guido van Rossum

Created Python in 1989 while working at CWI in the Netherlands.

- Named after "Monty Python's Flying Circus".
- Known as the "Benevolent Dictator for Life" (BDFL) until 2018.
- Philosophy: "Beautiful is better than ugly."



Versatility: Cloud SDKs, Automation, API interaction, and Web Apps.

Readability: Clear syntax makes it "Executable Pseudocode".

Ecosystem: Massive library support (Boto3, Requests, Pandas).



Python: Modular and high level programming language
Bash: System level scripting language

Features	Bash	Python
Primary Use	System tasks, command orchestration	General-purpose programming, data manipulation, complex automation
Availability	Often requires installation and dependency management	Often requires installation and dependency management
Syntax	Concise for shell commands, can become complex for logic	Highly readable, enforces structure, consistent across complexity levels
Data Structures	Limited (basic arrays and streams)	Rich data types (lists, dicts, objects)
Libraries	Leverages external system utilities (grep, awk, sed)	Vast internal and third-party package ecosystem
Error Handling	Limited, debugging can be difficult	Robust, with exceptions and debugging tools



In Python, indentation is not optional - it defines the code block structure.

- Standard: 4 spaces per level.
- No curly braces {} or begin/end needed.
- Case sensitive: DevOps != devops.

```
if 5 > 2:  
    print("Five is greater than two!")
```

```
if 5 > 2:  
print("Five is greater than two!")
```




Variables are containers for storing data values. Python is **Dynamically Typed**.

Naming: Use snake_case for variables and UPPER_CASE for constants.

Legal variable names:

```
myvar = "John"  
my_var = "John"  
_my_var = "John"  
myVar = "John"  
MYVAR = "John"  
myvar2 = "John"
```

Illegal variable names:

```
2myvar = "John"  
my-var = "John"  
my var = "John"
```




- In programming, data type is an important concept.
- Variables can store data of different types, and different types can do different things.
- Python has the following data types built-in by default, in these categories:

Text Type:

`str`

Numeric Types:

`int`, `float`, `complex`

Sequence Types:

`list`, `tuple`, `range`

Mapping Type:

`dict`

Set Types:

`set`, `frozenset`

Boolean Type:

`bool`

Binary Types:

`bytes`, `bytearray`, `memoryview`

None Type:

`NoneType`

What will this print?

```
x = 5  
print(type(x))
```



Strings in Python are surrounded by either single quotation marks, or double quotation marks.

- 'hello' is the same as "hello".
- You can display a string literal with the print() function:

Loop through the letters in the word "banana":

```
for x in "banana":  
    print(x)
```

Check if "free" is present in the following text:

```
txt = "The best things in life are free!"  
print("free" in txt)
```

The len() function returns the length of a string:

```
a = "Hello, World!"  
print(len(a))
```

Check if "expensive" is NOT present in the following text:

```
txt = "The best things in life are free!"  
print("expensive" not in txt)
```



Lists are used to store multiple items in a single variable.

- Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage.
- Lists are created using square brackets: []

Create a List:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist)
```

Print the number of items in the list:

```
thislist = ["apple", "banana", "cherry"]  
print(len(thislist))
```

A list with strings, integers and boolean values:

```
list1 = ["abc", 34, True, 40, "male"]
```

Using the `list()` constructor to make a List:

```
thislist = list(("apple", "banana", "cherry"))  
print(thislist)
```



- Dictionaries are used to store data values in key:value pairs.
- A dictionary is a collection which is ordered*, changeable and do not allow duplicates.

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}
```

Print the "brand" value of the dictionary:

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
print(thisdict["brand"])
```

String, int, boolean, and list data types:

```
thisdict = {  
    "brand": "Ford",  
    "electric": False,  
    "year": 1964,  
    "colors": ["red", "white", "blue"]  
}
```



Python supports the usual logical conditions from mathematics:

- Equals: `a == b`
- Not Equals: `a != b`
- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
- Greater than or equal to: `a >= b`

These conditions can be used in several ways, most commonly in "if statements" and loops.

An "if statement" is written by using the **if** keyword.

If statement:

```
a = 33
b = 200
if b > a:
    print("b is greater than a")
```

Using a boolean variable:

```
is_logged_in = True
if is_logged_in:
    print("Welcome back!")
```

If statement, without indentation (will raise an error):

```
a = 33
b = 200
if b > a:
print("b is greater than a") # you will get an error
```





Testing multiple conditions:

```
score = 75

if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
```

```
a = 200
b = 33
if b > a:
    print("b is greater than a")
else:
    print("b is not greater than a")
```

One-line `if / else` that prints a value:

```
a = 2
b = 330
print("A") if a > b else print("B")
```




A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)
```

Loop through the letters in the word "banana":

```
for x in "banana":  
    print(x)
```

Do not print banana:

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    if x == "banana":  
        continue  
    print(x)
```

Exit the loop when **x** is "banana":

```
fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)  
    if x == "banana":  
        break
```





Using the range() function:

```
for x in range(6):  
    print(x)
```

Using the start parameter:

```
for x in range(2, 6):  
    print(x)
```

Increment the sequence with 3 (default is 1):

```
for x in range(2, 30, 3):  
    print(x)
```

Break the loop when **x** is 3, and see what happens with the else block:

```
for x in range(6):  
    if x == 3: break  
    print(x)  
else:  
    print("Finally finished!")
```



Print each adjective for every fruit:

```
adj = ["red", "big", "tasty"]
fruits = ["apple", "banana", "cherry"]

for x in adj:
    for y in fruits:
        print(x, y)
```

```
for x in [0, 1, 2]:
    pass
```

for loops cannot be empty, but if you for some reason have a for loop with no content, put in the pass statement to avoid getting an error.



With the while loop we can execute a set of statements as long as a condition is true.

Print i as long as i is less than 6:

```
i = 1
while i < 6:
    print(i)
    i += 1
```

Print a message once the condition is false:

```
i = 1
while i < 6:
    print(i)
    i += 1
else:
    print("i is no longer less than 6")
```

Also has both break and continue clauses.



- Comments can be used to explain Python code.
- Comments can be used to make the code more readable.
- Comments can be used to prevent execution when testing code.

```
#This is a comment  
print("Hello, World!")
```

```
print("Hello, World!") #This is a comment
```

```
""""  
This is a comment  
written in  
more than just one line  
""""  
print("Hello, World!")
```



- A function is a block of code which only runs when it is called.
- A function can return data as a result.
- A function helps avoiding code repetition.

```
def my_function():  
    print("Hello from a function")  
  
my_function()
```

Function names follow the same rules as variable names in Python:

- A function name must start with a letter or underscore
- A function name can only contain letters, numbers, and underscores
- Function names are case-sensitive (myFunction and myfunction are different)



With functions - reusable code:

```
def fahrenheit_to_celsius(fahrenheit):  
    return (fahrenheit - 32) * 5 / 9  
  
print(fahrenheit_to_celsius(77))  
print(fahrenheit_to_celsius(95))  
print(fahrenheit_to_celsius(50))
```

A function that returns a value:

```
def get_greeting():  
    return "Hello from a function"  
  
message = get_greeting()  
print(message)
```

A function with one argument:

```
def my_function(fname):  
    print(fname + " Refsnes")  
  
my_function("Emil")  
my_function("Tobias")  
my_function("Linus")
```

```
def my_function(name): # name is a parameter  
    print("Hello", name)  
  
my_function("Emil") # "Emil" is an argument
```




Questions?

- Homework 5: CODING!!!!
- Next Lesson: Python for DevOps Part 2.
- Prep Work: Ensure Python 3 is installed on your local machines.

Questions?

CONTACT US